DESIGN OF EXPERIMENTS × GPU KERNELS

Hardware-aware autotuning of a CUDA GEMM kernel with *response surface methodology*

ABSTRACT

A single parameterised CUDA matrix-multiplication kernel, adapted from a public tutorial kernel, was tuned over six compile-time factors on an NVIDIA H200, using a 2^{6-1} fractional-factorial screen, a 6-factor central composite design, a second-order response surface, constrained sequential quadratic programming and measured validation. Every result is checked against an exhaustive sweep of all 1,396 feasible configurations. **The pipeline selected the second-best configuration of 1,396 (0.17% from the optimum) after measuring 59.** Random search with the same budget did as well in 7.8% of 10,000 trials and needs a median of 424 evaluations to match it. The pre-registered second-order surface was statistically inadequate (lack of fit 7.8% against a 5% limit), because of 3-factor interactions and abrupt compiler effects, so the final choice was made by measurement. The winner was a corner of the screening design, so the gain comes from screening and from where its levels were placed, not from the fitted surface. The selected kernel reaches 72.0% of cuBLAS FP32 throughput and is 1.31× faster than the default tile sizes with the inner loop fully unrolled.

0.17%	59	1.31×	72.0%
gap to the exhaustive optimum	configurations measured, of 1,396	faster than the full-unroll default	of cuBLAS FP32 throughput

AUTHOR
Vaidheesh Bharadwaj

ROLL NO.
230957088

GUIDE
Prof. Sunita Sharma

PLATFORM
NVIDIA H200 · CUDA 13.0

01 Introduction and research questions

> The same kernel source runs at 26.1 TFLOPS with small default tiles, 30.5 TFLOPS with the parameters of the tutorial it comes from and 34.3 TFLOPS with tuned ones: six compile-time numbers decide a 1.31× difference.

Choosing those numbers is usually done by exhaustive search, which grows with every new factor, by search-based autotuners such as OpenTuner, CLTune and Kernel Tuner, or by machine-learning autotuners such as TVM and Ansor, which need many samples. This project tests whether classical Design of Experiments (DOE) and Response Surface Methodology (RSM) can find a near-best configuration from a small, statistically designed sample, as proposed in the synopsis.

TABLE 1 Research questions. RQ3 is the primary claim; RQ5 is context.

	QUESTION
RQ1	Can a fractional factorial design separate the parameters that matter from noise?
RQ2	Does a second-order response surface capture the GPU's non-linear behaviour?
RQ3	With the same number of trials, does DOE–RSM beat random search?
RQ4	How close does DOE–RSM get to the exhaustive-search optimum?
RQ5	What share of cuBLAS throughput does the tuned kernel reach?

02 Setup and measurement protocol

> One parameterised kernel was tuned over six factors, with every rule of the experiment fixed in a written contract before any final data was collected.

Kernel and design space

The kernel is a 2D block-tiled shared-memory SGEMM computing C = A·B (row-major, FP32 inputs and accumulation, CUDA-core FFMA only, no Tensor Cores). It is adapted from kernel 5 ("2D blocktiling") of Boehm's public SGEMM worklog; this project added the compile-time parameterisation, the unroll factor, the feasibility assertions and a fault-injection switch for testing the correctness check. The six factors are injected as compile-time macros.

TABLE 2 Tunable factors and their levels in the full design space

FACTOR	MEANING	LEVELS
BM	block output-tile rows (shared memory)	32 · 64 · 128 · 256
BN	block output-tile columns (shared memory)	32 · 64 · 128 · 256
BK	K-depth of each shared-memory tile	8 · 16 · 32 · 64
TM	output rows per thread (register tile)	2 · 4 · 8
TN	output columns per thread (register tile)	2 · 4 · 8
UNROLL	unroll factor of the inner dot-product loop	1 · 2 · 4 · 8

Of the 2,304 combinations, **1,396 are feasible**. The rest break a hardware or tiling constraint: more than 1,024 threads per block (528), shared memory over 48 KiB (168), a tile-load overflow (140) or a fractional load stride (72). The feasibility filter was checked against the compiler on all 2,304 candidates with no disagreements, and the kernel carries the same rules as compile-time assertions.

Platform

NVIDIA H200 (sm_90, 132 SMs), driver 595.71.05, CUDA 13.0.88, cuBLAS 13.1.1.3, `nvcc -O3`. The GPU is shared with other users' processes.

Measurement protocol

- **Timing.** Each launch is timed individually with a CUDA event pair; allocation, transfers, verification and compilation are outside the timed region. A measurement block is the median of 30 launches after 10 warm-up launches at 4096^3 (80 and 25 at 2048^3; 300 and 100 at 1024^3).
- **Clock.** The SM clock was locked for the whole run; the driver held 1830 MHz (1980 MHz was requested) on all 2,001 measurements. A stability gate measured two configurations in 10 separated blocks: coefficient of variation 0.033% and 0.106%, against a 1% limit.
- **Randomisation.** Every experiment ran as replicate blocks, each a fresh random permutation of the design, so replicates of one point never ran back to back.
- **Shared GPU.** NVML per-process accounting checked every measurement window; a window in which another process used the SMs was repeated, up to six attempts. 37 measurements were repeated and 1 remained flagged, kept and marked as the contract requires.
- **Correctness.** Every configuration was checked against cuBLAS once per shape by relative Frobenius error, with a tolerance of 10^{-5} fixed before any data; all passed. A deliberately broken kernel produced an error of 0.044, matching the predicted $\sqrt{(8/4096)}$, and both the custom kernel and cuBLAS agree with a float64 CPU reference to within 6×10^{-7}.

03 Screening (RQ1)

> A 32-run fractional factorial found all six factors significant at 4096^3 ($R^2 = 0.959$). The unroll factor dominates, with 54% of the variance including its interactions.

Design. A 2^{6-1} half fraction with generator F = ABCDE (defining relation I = ABCDEF, resolution VI): main effects are aliased only with 5-factor interactions and 2-factor interactions only with 4-factor interactions. Each factor ran at two legal levels (BM, BN 64/128; BK 8/32; TM, TN 4/8; UNROLL 1/8). There are no centre points, because the midpoint of a power-of-two pair is not a legal configuration; pure error comes from 3 replicate blocks (96 measurements per shape).

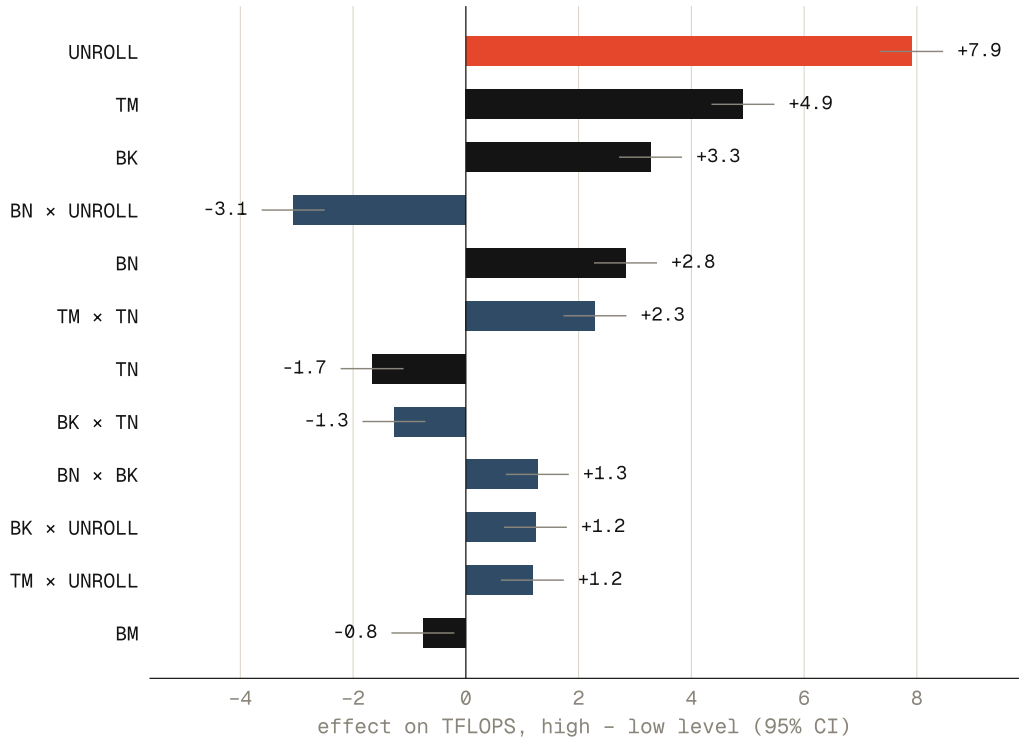


FIGURE 1 Screening effects at 4096^3 with 95% confidence intervals: six main effects and the six largest 2-factor interactions. All are significant.

Raising UNROLL from 1 to 8 adds 7.9 TFLOPS on average and TM from 4 to 8 adds 4.9; the strongest interaction, BN × UNROLL, is negative (−3.1). UNROLL's dominance partly reflects its low level: UNROLL = 1 forbids unrolling the inner loop at all, so much of its effect is the cost of switching unrolling off rather than a gain found by tuning. The pre-registered retention rule (keep a factor if its main effect or any interaction containing it has $p < 0.05$) kept all six; the weakest, BM, still had $p = 0.0026$.

Diagnostics. Randomising the run order removed the autocorrelation that a fixed order produced in a pilot run: the Durbin–Watson statistic on run order is 1.86, against 0.60 in the pilot. There is no drift between blocks ($p = 1.0$) and replicate noise is small (median coefficient of variation 0.037%). Residual variance is not constant (Breusch–Pagan $p < 0.001$), but refitting on log latency gives the same significant factors at all three shapes. Because measurement noise is so small, the residuals are mostly real 3-factor interactions the model omits; that inflates the error estimate, so the tests are conservative.

Effect of matrix shape. The same screen was repeated at 2048^3 and 1024^3 (Figure 2). Factor rankings at 4096^3 and 2048^3 agree exactly (Kendall $\tau = 1.0$); at 1024^3 they do not ($\tau = -0.07$): BM rises from 1.9% to 23.6% of the variance and TM falls from 22.7% to 3.2%. A likely cause is GPU fill: at 1024^3 a 128 × 128 tile yields only 64 thread blocks for the H200's 132 SMs, so tile size decides how much of the GPU is used. Tuned settings should not be carried from large to small matrices.

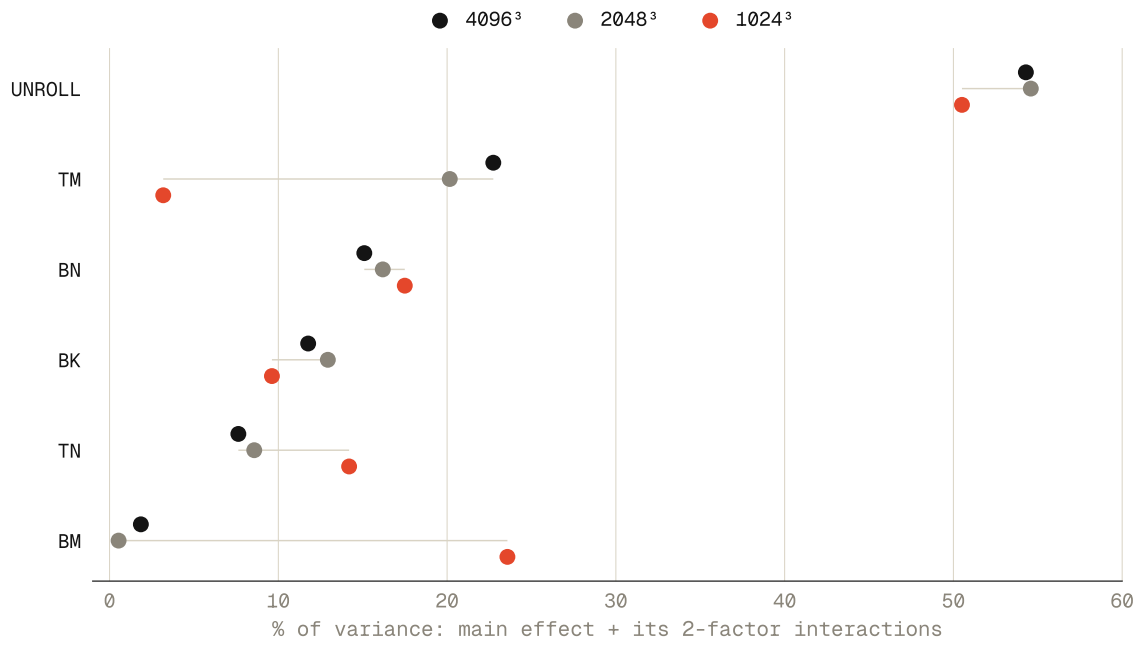


FIGURE 2 Share of variance per factor (main effect plus its 2-factor interactions) at three matrix sizes.

04 Response surface (RQ2)

> A second-order surface explains 95.6% of the variance but fails the pre-registered adequacy test: its lack of fit is 7.8% of the model mean square against a 5% limit. The answer to RQ2 is “close, but not adequate”.

Design. A 6-factor central composite design in $\log_2$-coded units: the 32-run screening design as its factorial core, 12 axial points at the rotatable $\alpha = 2.378$ and 4 centre points. Each coded point was mapped to the nearest legal power of two (ties to the lower level) and the model was fitted on the realised coded values, so the centre runs as (64, 64, 16, 4, 4, 2). In total, 48 points × 3 randomised blocks = 144 measurements on 45 distinct configurations.

TABLE 3 Response-surface models. Lack of fit is the lack-of-fit mean square as a percentage of the model mean square; held-out error leaves out one configuration (all its replicates) at a time.

MODEL	PARAMS	R^2	PRED. R^2	LACK OF FIT	GATE (≤ 5%)	HELD-OUT ERROR, MEDIAN / MAX
Second order · pre-registered	30	0.956	0.932	7.8%	fails	10.6% / 46%
Second order · log latency (fallback)	30	0.961	0.940	7.0%	fails	11.4% / 38%
Second order + 3-factor terms	40	0.982	0.970	10.1%	fails	32.6% / 412%

The lack-of-fit F-test is overwhelming ($p < 10^{-150}$) because replicate noise is only about 0.05%, so even small systematic errors are detectable. 80% of the misfit lies inside the factorial core rather than at the axial extremes, and canonical analysis shows the fitted quadratic is a saddle with no interior maximum.

The 3-factor investigation

After the gate failed, the project owner chose to report the failure and then test the obvious remedy: adding the 10 estimable 3-factor interaction pairs (the half-fraction core aliases each triple with its complement). Six are significant, the strongest being BM·BN·BK = TM·TN·UNROLL ($p = 1.4\times10^{-13}$), and together they explain 59% of the quadratic's residual. The augmented model still fails the gate and is overfitted: 40 parameters on 45 distinct configurations make its error on a held-out configuration three times worse. The ordinary predicted R^2 hides this, because it leaves out one measurement at a time while the other replicates of that configuration stay in the training set.

Part of the misfit is not polynomial at all. For (BM, BN, BK, TM, TN) = (64, 64, 32, 8, 4), throughput is non-monotonic in the unroll factor (Figure 3). A smooth surface cannot follow a compiler code-generation jump like that.

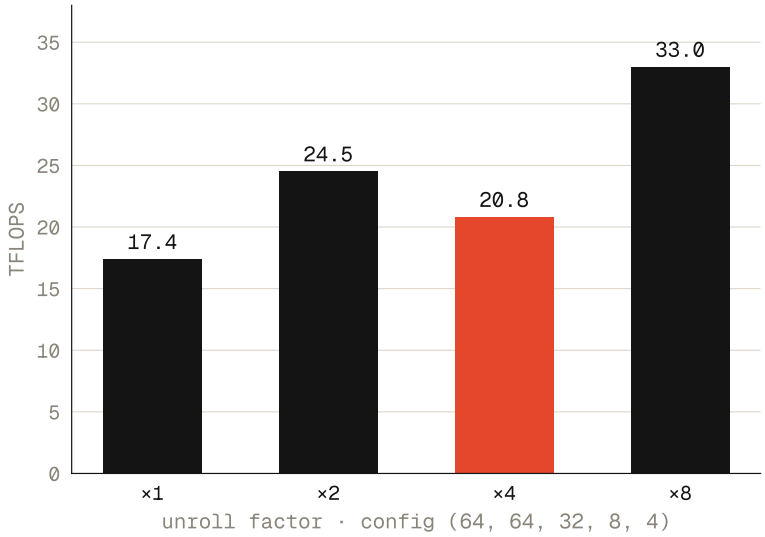


FIGURE 3 Throughput against unroll factor for configuration (64, 64, 32, 8, 4): unrolling by 4 is slower than by 2.

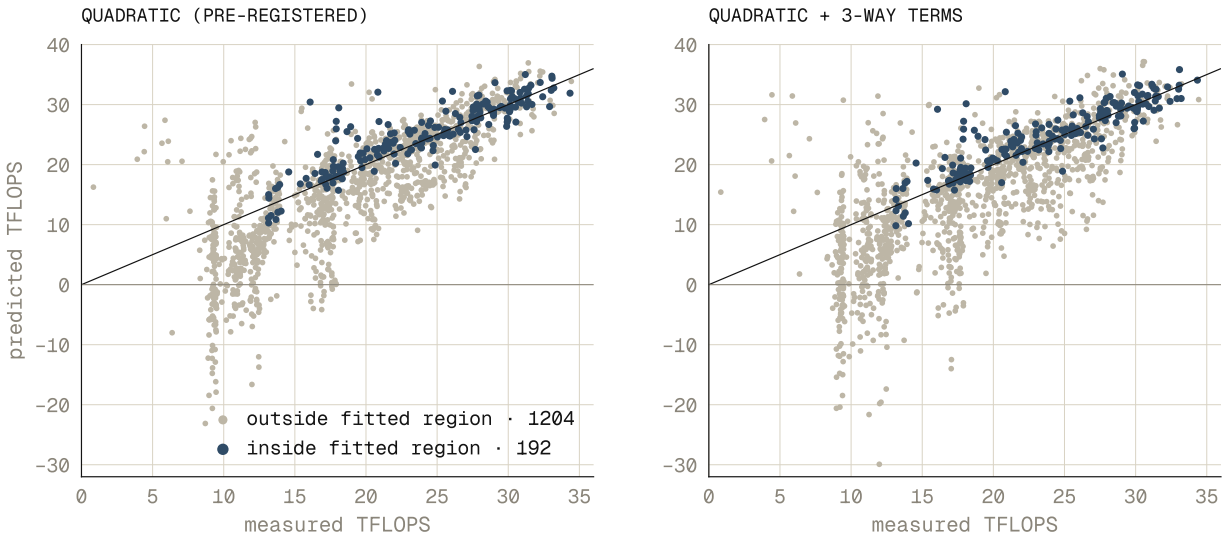


FIGURE 4 Predicted against measured throughput for all 1,396 feasible configurations, for both models. Blue: inside the jointly tested region; grey: outside.

Checked against the exhaustive sweep (Figure 4), both models are usable inside the region they were fitted on (median error 5.5% for the quadratic, 5.0% augmented) and unreliable outside it (21.8% and 22.7%), sometimes predicting negative throughput. The model's best configuration outside the fitted region was predicted at 37.1 TFLOPS and measured at 30.5. This is why optimisation was restricted to the fitted region. The residuals (Figure 5) show no run-order structure (Durbin–Watson 2.15) and no influential point (largest Cook's distance 0.049): the failure is systematic misfit, not noise or outliers.

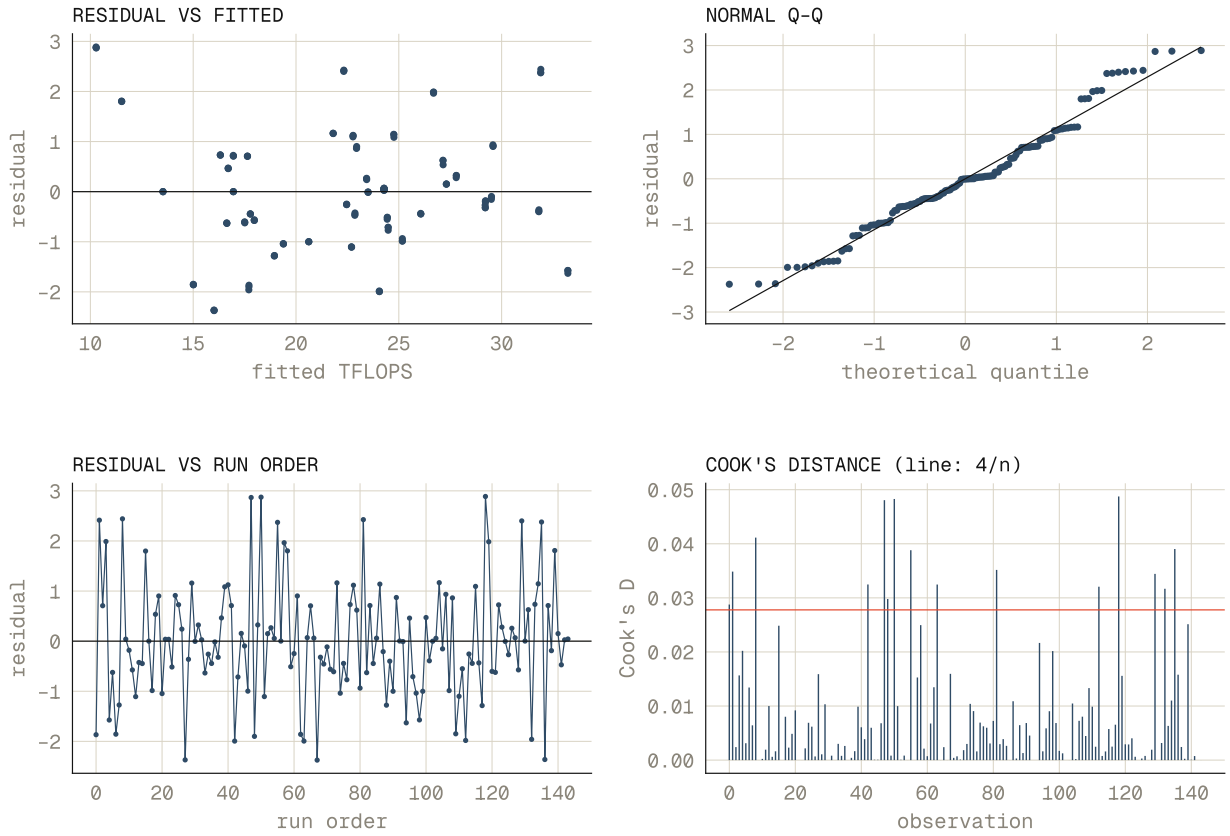


FIGURE 5 Residual diagnostics for the pre-registered second-order model.

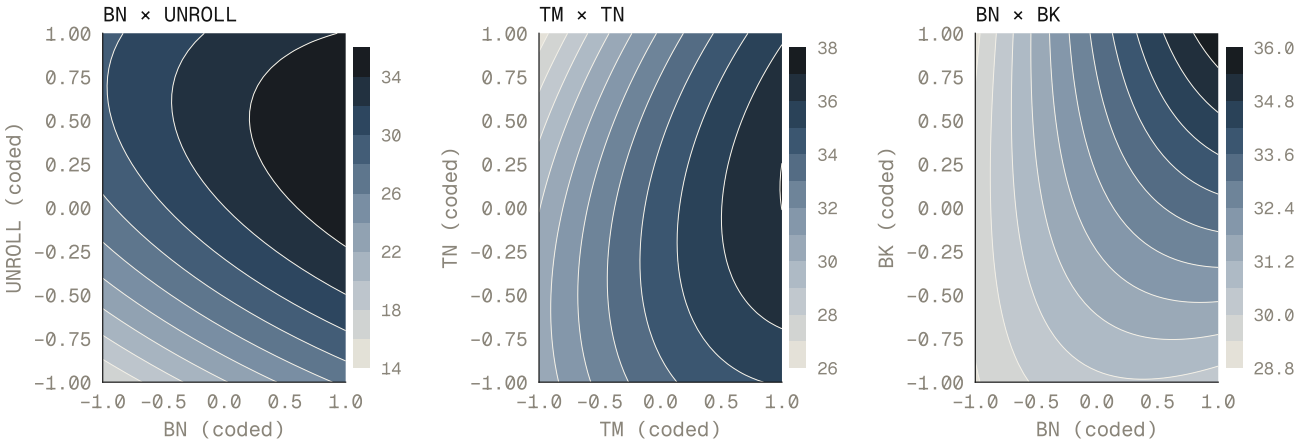


FIGURE 6 Fitted surface for the three strongest interactions (augmented model; it fails the adequacy gate and is shown for interpretation only). Other factors are held at the model's best configuration.

05 Optimisation and validation

> Measured validation, not the model, picked the final configuration: (128, 128, 32, 8, 8, 8) at 34.32 TFLOPS, which neither model ranked first.

Optimisation was limited to the region the factorial core tested jointly (every coded factor within [−1, 1]), which leaves 192 feasible lattice configurations. Two methods proposed candidates: sequential quadratic programming (SciPy SLSQP, 30 random starts, all converged) on the continuous relaxation, with the thread-count, shared-memory and tile-load limits as nonlinear constraints, then projected to the lattice with its feasible ±1-level neighbourhood; and exhaustive evaluation of each fitted model over the 192 lattice configurations. The augmented model's top 5, the quadratic's top 3 and the SQP neighbourhood gave 15 distinct candidates, each measured in 5 randomised blocks.

TABLE 4 Validation candidates: the five fastest, each model's first choice and the largest miss (15 candidates in total, 5 blocks each).

CONFIGURATION	MEASURED TFLOPS (SD)	AUGMENTED PRED.	QUADRATIC PRED.	RANK: MEAS.	AUG.	QUAD.
(128, 128, 32, 8, 8, 8) · selected	34.32 (0.04)	34.07	31.90	1	3	11
(128, 128, 32, 8, 8, 4)	33.08 (0.01)	35.83	34.39	2	1	3
(64, 128, 32, 8, 8, 4)	33.05 (0.03)	33.23	34.73	3	6	2
(64, 64, 32, 8, 4, 8)	33.02 (0.02)	32.57	32.37	4	11	8
(64, 128, 16, 8, 8, 4)	32.21 (0.02)	32.95	33.69	5	9	4
(64, 128, 32, 8, 4, 4)	31.17 (0.03)	33.07	35.01	8	8	1
(64, 64, 32, 8, 4, 4)	20.84 (0.01)	32.15	32.07	15	13	10

Within this narrow top band both models rank poorly (Spearman correlation with measured throughput 0.25 augmented, 0.37 quadratic; mean prediction error 9.0% and 8.5%). Trusting a model's own first choice would have cost 3.6% (augmented) or 9.2% (quadratic); benchmarking a candidate set instead of a single rounded optimum made the difference.

Against the exhaustive optimum. The true best configuration, (64, 128, 64, 8, 8, 8) at 34.40 TFLOPS, has BK = 64, outside the region the models were allowed to search, so the restriction chosen to avoid extrapolation also put it out of reach. The top 20 exhaustive configurations were re-measured in 3 more blocks: the selected configuration is second, 0.17% slower, a borderline-significant difference (Welch $p = 0.042$); the third is 33.22 TFLOPS.

06 Sample efficiency (RQ3, RQ4)

> With 59 evaluations DOE–RSM landed 0.17% from the optimum; random search needs a median of 424 evaluations to do as well.

DOE–RSM consumed 59 unique configurations: 32 in screening, 13 new ones in the CCD and 14 new ones in validation. Random search was given exactly that budget, repeated 10,000 times by sampling from the exhaustive measurements of all 1,396 configurations.

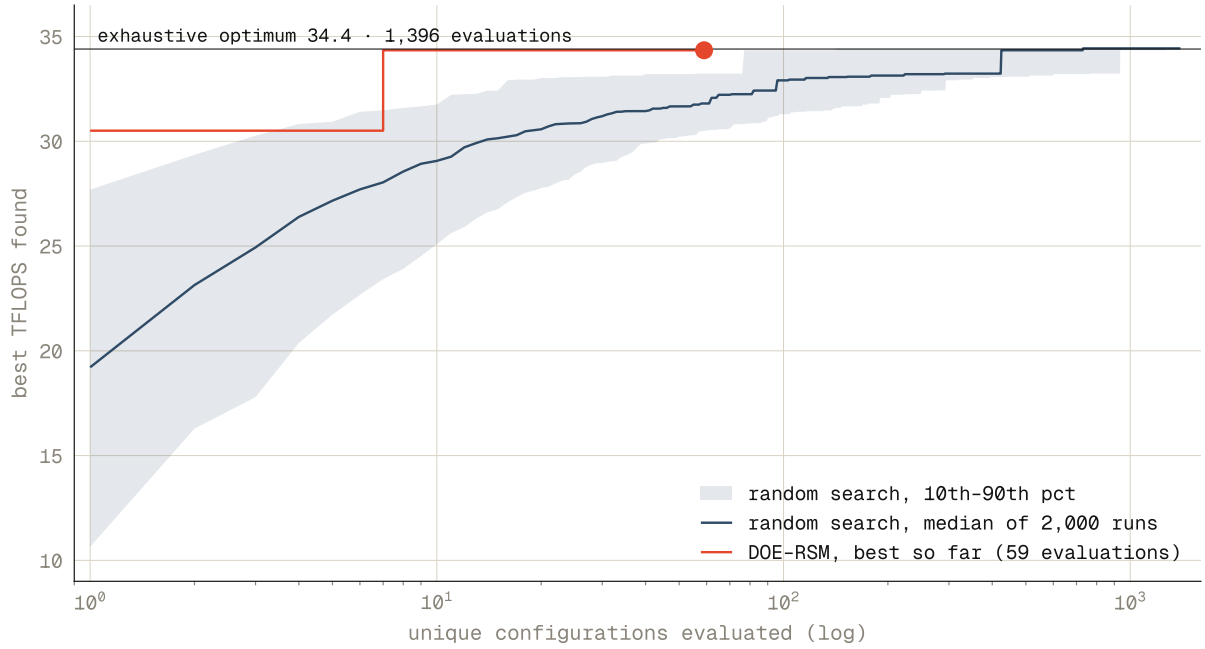


FIGURE 7 Best throughput found against configurations evaluated (log scale). DOE-RSM in vermilion; random search median and 10th-90th percentile band in blue.

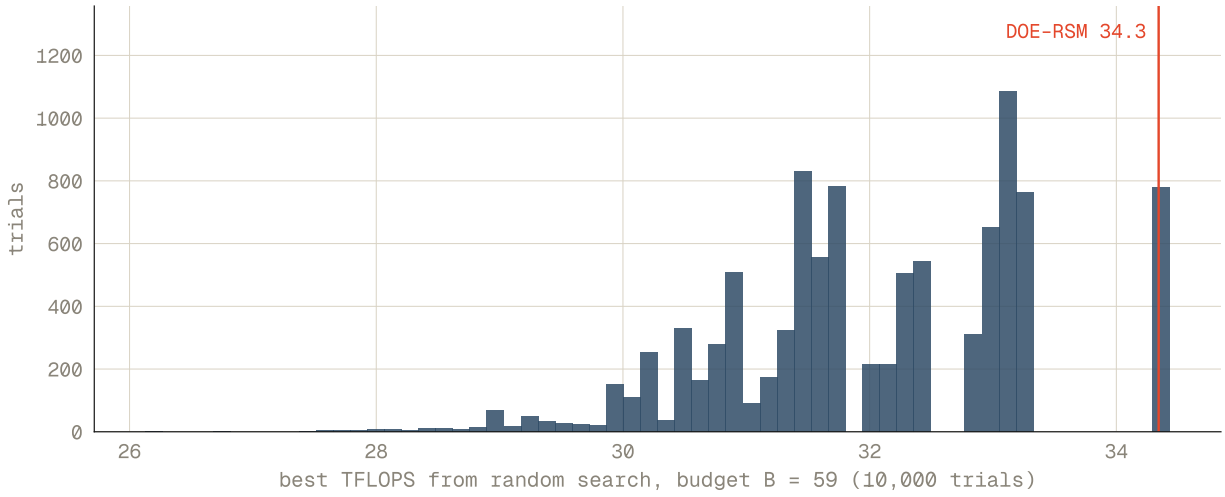


FIGURE 8 Distribution of random search's best result at the same 59-evaluation budget, 10,000 trials.

TABLE 5 Equal-budget comparison. Gaps are in latency against the exhaustive optimum (3.995 ms); single-configuration rows are from the exhaustive sweep.

METHOD	EVALUATIONS	BEST TFLOPS	GAP TO OPTIMUM
DOE–RSM (this project)	59	34.34	0.17%
Random search, same budget (median; 10th–90th pct)	59	32.07 (30.50-33.23)	7.26%
Exhaustive search	1,396	34.40	0%
Tutorial parameters (128, 128, 8, 8, 8, 8), one informed guess	1	30.54	12.7%
Full-unroll default (64, 64, 8, 4, 4, 8)	1	26.13	31.7%
Unroll-disabled default (64, 64, 8, 4, 4, 1)	1	13.15	161%

Random search matched or beat DOE–RSM in 7.8% of trials and needs a median of 424 evaluations (10th–90th percentile 77–936), about 7× the budget, to reach the same result.

What to credit to which stage. The selected configuration is the all-high corner of the screening design and was first measured at evaluation 7. The response surface and validation confirmed it, by measuring it fastest among 15 model-proposed candidates, but did not discover it. Its presence in the screening design follows from where the screening levels were placed (by engineering judgement, before any measurement). The efficiency gain belongs to the design as a whole, screening included, not to the fitted surface alone.

A fairer reference than random search. Random search starts with no knowledge of GPUs, while the screening levels were chosen by someone who has it, so beating random search does not show that the statistical method adds value beyond that knowledge. A better comparison is an informed engineer. Their natural first guess, the tutorial's own parameters, reaches 30.54 TFLOPS (rank 51 of 1,396) in one evaluation; the all-high screening corner, an equally natural guess, is the selected configuration itself. What this study supports is that a designed screen around sensible levels finds a near-optimal configuration cheaply and explains which factors matter. It does not show that the response surface found it.

07 Comparison with cuBLAS (RQ5)

> The selected kernel attains 72.0% of cuBLAS FP32 throughput. It is 1.31× faster than the default tile sizes with the inner loop fully unrolled, and 1.12× faster than the tutorial's parameters.

Each configuration was benchmarked in the same process as cuBLAS, alternating 30-launch blocks of each (10 per process, 3 randomised processes: 900 launches per side). cuBLAS ran in pedantic FP32 mode with TF32 disabled; its default mode gave identical times, and its output matches the custom kernel's bit for bit, so the comparison is FP32 against FP32.

TABLE 6 Interleaved comparison with cuBLAS, 4096^3, pedantic FP32.

CONFIGURATION	MEDIAN LATENCY (MS)	CUSTOM TFLOPS	CUBLAS TFLOPS	SHARE OF CUBLAS
DOE–RSM selection (128, 128, 32, 8, 8, 8)	4.001	34.35	47.71	72.0%
Exhaustive best (64, 128, 64, 8, 8, 8)	3.993	34.42	47.68	72.2%
Unroll-disabled default (64, 64, 8, 4, 4, 1)	10.447	13.16	47.69	27.6%

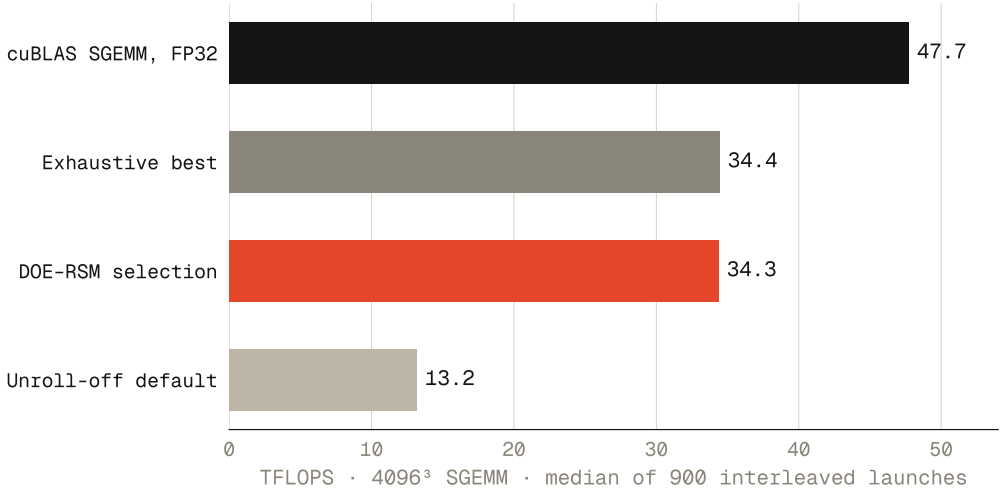


FIGURE 9 Throughput of the unroll-disabled default, the DOE-RSM selection, the exhaustive best and cuBLAS.

Choice of baseline. The default in the table has UNROLL = 1, which forbids unrolling the inner loop. Against it the selection is 2.61× faster, but most of that gap is the cost of switching unrolling off: the same tiles with UNROLL = 8, which unrolls the loop completely at BK = 8, already reach 26.13 TFLOPS (54.8% of cuBLAS) in the exhaustive sweep. The 1.31× figure against that configuration is the honest measure of what tuning bought.

The remaining 28% is a ceiling of the kernel design, not of the tuning: even the best of all 1,396 configurations stops at 72.2%. cuBLAS uses techniques this kernel does not have (vectorised 128-bit loads, asynchronous double-buffered copies, warp-level tiling); adding them would change the kernel family and lies outside this study.

08 Register spilling

> Register spilling is rare in this kernel family and does not drive its non-linear behaviour: 28 of 1,396 configurations spill, and they are not systematically slower.

The compiler's resource report was recorded for every configuration. Register use ranges from 20 to 255 per thread. Spilling configurations have a median of 21.6 TFLOPS against 19.0 for the rest; most sit at 64 registers, the per-thread limit a 1,024-thread block imposes. Throughput rises with registers per thread (Spearman 0.47) and falls with occupancy (–0.45): larger register tiles reuse each loaded value more, which outweighs having fewer threads resident. The synopsis expected the surface to reveal register spilling; in this kernel the non-linearity comes instead from register-tile reuse, unroll-dependent code generation and, at small shapes, GPU fill.



FIGURE 10 Registers per thread against throughput for all 1,396 feasible configurations; spilling configurations in vermilion.

09 Deviations and threats to validity

> One deviation from the pre-registered contract was made after seeing data; everything else ran as written.

Deviation. The contract said not to optimise a surface that fails the adequacy gate. After it failed, the project owner chose to report the failure (Section 4) and continue with the 3-factor-augmented model as a proposal generator, selecting the final configuration only by measured validation. Both models' recommendations were benchmarked, so the effect of that choice is visible in Table 4.

- **Shared GPU.** Contested measurement windows were detected through NVML and repeated (37 repeats, 1 still flagged). NVML samples utilisation periodically and can miss very short bursts; the median over 30 launches limits their influence.
- **Clock.** The requested 1980 MHz lock was held at 1830 MHz by the driver. Every measurement ran at that clock, so all throughputs, cuBLAS included, are about 7.5% below unlocked boost figures; ratios are unaffected.
- **Scope.** One GPU, FP32 on CUDA cores, one kernel family and one primary shape (4096^3). The 1024^3 screen shows factor importance changes with shape, so the selected configuration should not be carried to other shapes without re-tuning.
- **Power-of-two lattice.** Coded design points were snapped to legal levels, so the CCD is not exactly rotatable and its centre runs at (64, 64, 16, 4, 4, 2); models were fitted on the realised levels.
- **Where the winner came from.** The selected configuration is a corner of the screening design, so the result depends partly on where the screening levels were placed, and the true optimum lies outside the region the models were allowed to search.
- **Baselines.** Random search is the weakest sensible baseline. The study has no Bayesian-optimisation or local-search comparison, and the unroll-disabled default overstates the gain from tuning (Section 7).

- **Ground truth.** Random search was scored with the single-block exhaustive measurements; the top 20 were re-measured in 3 more blocks and kept their order.
- **Correctness reference.** At 4096^3 cuBLAS sums in the same order as this kernel, so their outputs are identical; agreement with an independent float64 CPU reference was checked for the default and selected configurations at 512^3 and 1024^3.
- **Pilot data excluded.** An earlier pilot run without clock locking or randomisation shaped this protocol; none of its measurements are used here.

10 Conclusions and future work

TABLE 7 Answers to the research questions.

	ANSWER
RQ1	Yes. The fractional factorial separated all six factors (R^2 0.959); their importance shifts at 1024^3.
RQ2	Close but inadequate: lack of fit 7.8% against a 5% limit, caused by 3-factor interactions and compiler jumps.
RQ3	Yes against random search: 59 evaluations against a median of 424 for the same result. The gain comes from the screen and its expert-chosen levels, not the fitted surface.
RQ4	0.17% from the exhaustive optimum: the second-best of 1,396 configurations.
RQ5	72.0% of cuBLAS FP32 throughput; 1.31× the full-unroll default and 1.12× the tutorial's parameters.

The practical lesson is to use the response surface to propose candidates and let measurement decide. Four extensions follow: a full 2^6 factorial core, so 3-factor interactions can be estimated without overfitting; tuning per matrix shape, since importance shifts below 2048^3; a head-to-head comparison with Bayesian optimisation, the method modern autotuners use; and kernel features such as vectorised loads and double buffering as categorical factors, to raise the 72% ceiling.

A Reproducibility

Every number in this report comes from run `final_20261028` and can be regenerated from its raw files. The experimental contract (`contract.json`, SHA-256 prefix `857dc5fbcf29`) was frozen before data collection and its hash is stamped on the run.

TABLE 8 Data and code.

FILE	CONTENTS
`results/final_20261028/measurements.csv`	2,001 measurement blocks: configuration, validity, clock, temperature, contention flags, registers, median and IQR latency
`results/final_20261028/raw_launches.csv`	every individual timed launch
`results/final_20261028/decisions/`	each automatic decision: retained factors, CCD design, validation candidates, final selection
`results/final_20261028/analysis/`	ANOVA tables, model coefficients, cross-validation, summaries
`results/feasibility_audit.csv`	all 2,304 candidates with feasibility status and the compiler's verdict
`cuda/gemm_kernel.cuh`, `cuda/bench.cu`	the kernel and its benchmark harness

The run is executed phase by phase with `python pipeline.py <run> <phase>` (stability, screening, ccd, validate, exhaustive, recheck, compare) and analysed with `python analysis.py <run> <phase>`.

B References

1. G. E. P. Box and K. B. Wilson, "On the Experimental Attainment of Optimum Conditions," *Journal of the Royal Statistical Society, Series B*, 1951.
2. D. C. Montgomery, *Design and Analysis of Experiments*, Wiley, 2017.
3. S. Boehm, "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog," 2022. Online: siboehm.com/articles/22/CUDA-MMM (kernel 5, source of the kernel used here).
4. J. Ansel et al., "OpenTuner: An Extensible Framework for Program Autotuning," PACT, 2014.
5. C. Nugteren and V. Codreanu, "CLTune: A Generic Auto-Tuner for OpenCL Kernels," MCSoC, 2015.
6. B. van Werkhoven, "Kernel Tuner: A Search-Optimizing GPU Code Auto-Tuner," *Future Generation Computer Systems*, 2019.
7. NVIDIA Corporation, *CUDA C++ Programming Guide*.
8. NVIDIA Corporation, *cuBLAS Library Documentation*.
9. D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan Kaufmann, 2016.
10. T. Chen et al., "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," OSDI, 2018.
11. L. Zheng et al., "Ansor: Generating High-Performance Tensor Programs for Deep Learning," OSDI, 2020.
12. J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, 2006.